



File System & Storage Management



Group Members

- **2022/ICT/23**
- **2022/ICT/50**
- **2022/ICT/96**
- **2022/ICT/110**
- **2022/ICT/117**
- **2022/ICT/118**
- **2022/ICT/128**



Content

- 01. Introduction to File Systems**
 - 02. File & Their Characteristics**
 - 03. File Structure & Access**
 - 04. File Attributes & Operations**
 - 05. Directory Systems**
 - 06. Disk Storage Management**
 - 07. Protection & Modern Files**
-

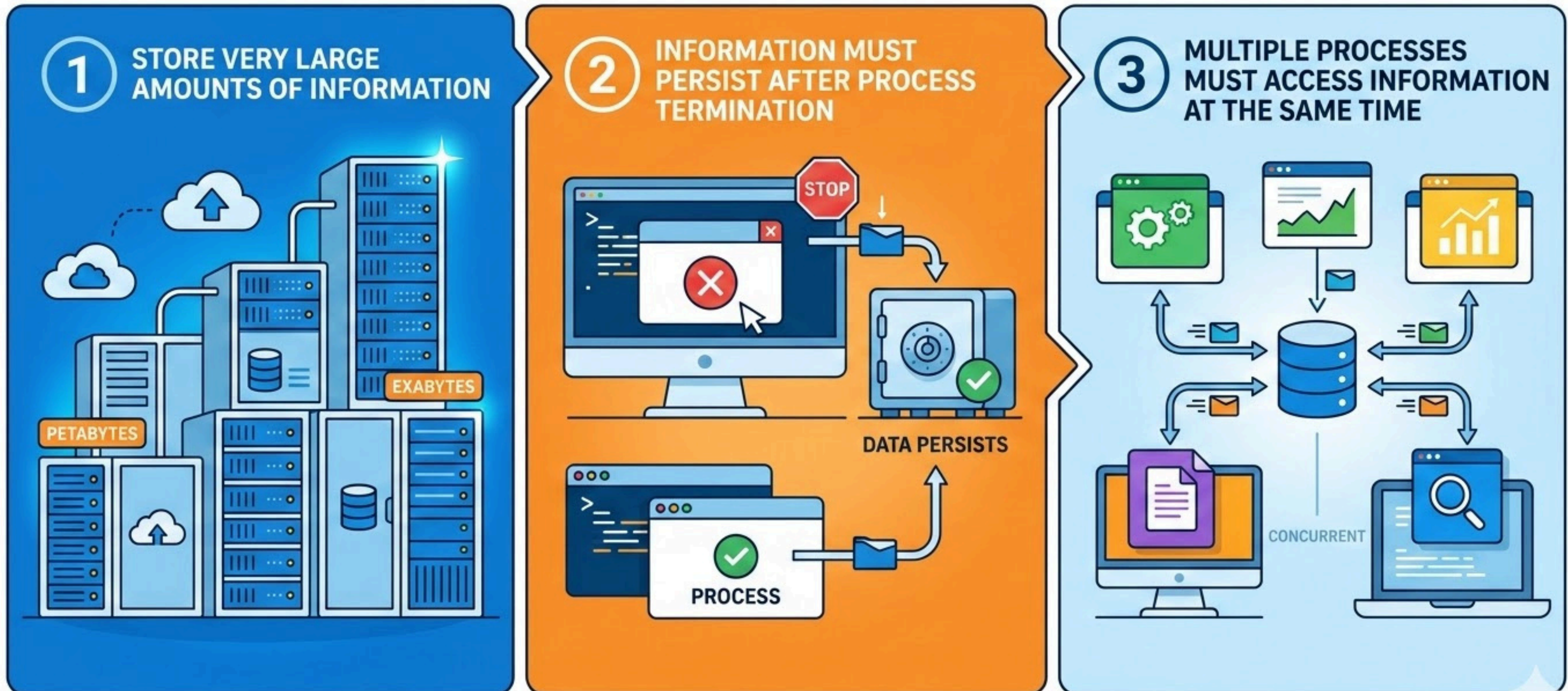
Need for File System

- Applications must store and retrieve large amounts of data.
- Process address space is limited in size.
- Data inside a process is lost when the process terminates.
- Data must survive system crashes.
- Multiple processes must access shared information concurrently.



Introduction

ESSENTIAL REQUIREMENTS FOR LONG-TERM STORAGE



What is a File?

- A file is a logical unit of information
- Created and managed by the Operating System (OS)
- Stored on secondary storage (like disk)
- Used for permanent data storage



File & Their Characterist

Logical View of a File



Sequence of Bytes (UNIX, Windows)



File = Continuous stream of bytes



Sequence of Records

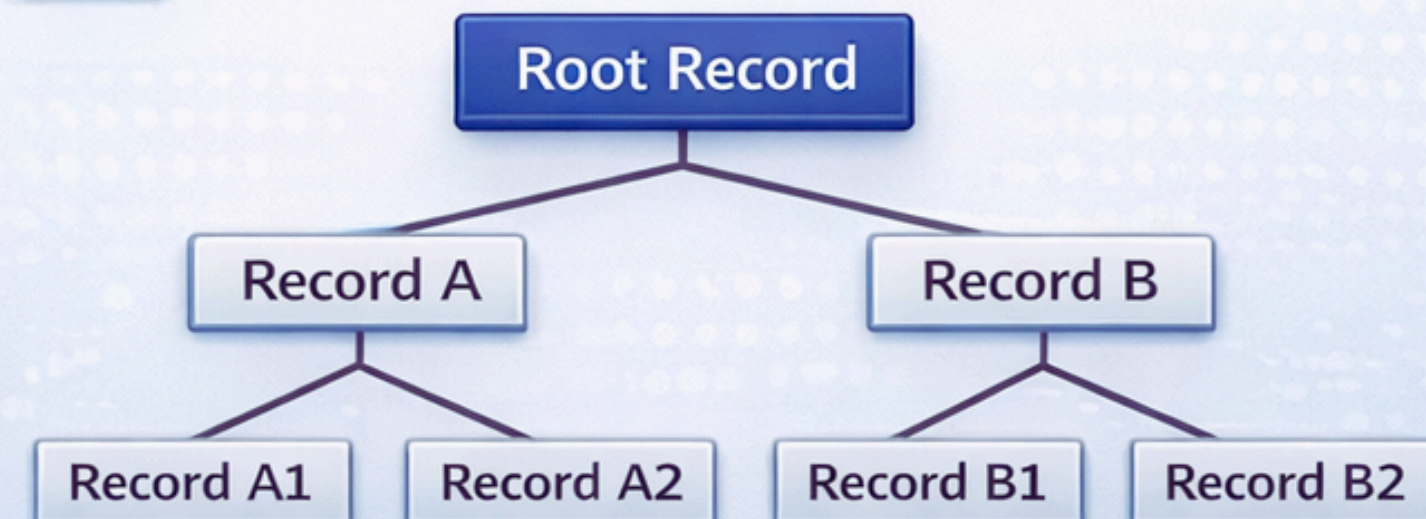


Record Example

ID	Name	Age



Structured Tree of Records



FILE NAMING

- When a file is created, it is given a name
 - Names can include letters, digits, and symbols
 - Length varies by OS
 - Case sensitivity:
 - UNIX → case sensitive (file ≠ File)
 - Windows → not case sensitive
-



Report.docx

File Extensions

- The part after the dot (.)
- Indicates file type
- Examples:
 - .txt → Text file
 - .c → C program
 - .jpg → Image
 - .pdf → Document
 - .exe → Executable file



Text file



C program



Image



Document



Executable file

File Structure Types

- **Byte Stream**

- File = sequence of bytes
- No internal structure
- Used in UNIX & Windows

- **Record-Based**

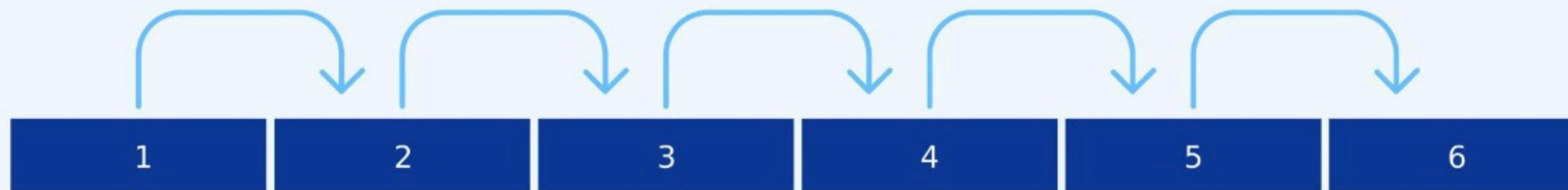
- File = collection of records
- Fixed-length or structured
- Used in older/mainframe systems



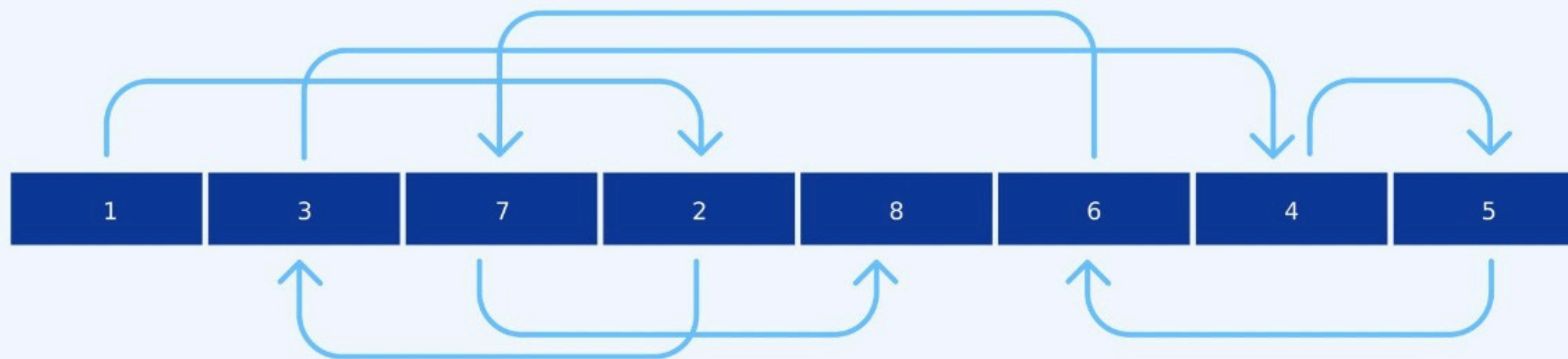
**File Structure
& Access**

Access Methods

Sequential Access



Random Access



1.Sequential Access

- Data read in order
- Start from beginning
- Cannot skip directly
 - Common in early systems

2.Direct (Random) Access

- Jump to any position
- Read data in any order
- Used in modern systems

➡ **Seek Operation**

Moves file pointer

Sets position for reading/writing

Performance Comparison

FEATURE	SEQUENTIAL ACCESS	RANDOM (DIRECT) ACCESS
Data Retrieval	MUST FOLLOW A STRICT LINEAR ORDER.	CAN JUMP TO ANY LOCATION INSTANTLY.
Speed	SLOW FOR FINDING SPECIFIC DATA IN LARGE FILES.	FAST FOR TARGETED DATA RETRIEVAL.
Efficiency	HIGHLY EFFICIENT FOR LARGE, CONTINUOUS READS.	EFFICIENT FOR "NEEDLE IN A HAYSTACK" TASKS.
Hardware Fit	IDEAL FOR MAGNETIC TAPES.	IDEAL FOR HDDs, SSDs, AND FLASH MEMORY.

File Attributes (Metadata)

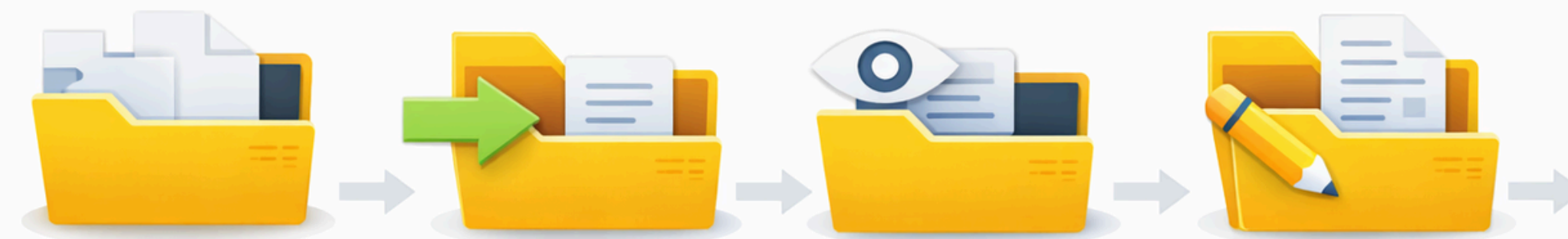
- File attributes are information about a file stored by the Operating System.
- They describe the file but are not part of the file's actual content.
- Common File Attributes
 - *File Name*
 - *File Type*
 - *Size*
 - *Location*
 - *Owner*
 - *Creation Date*
 - *Last Modified Date*
 - *Permissions*

File Attributes & Operations

File Permissions

- File permissions control access to files and define what actions users can perform.
- Common Permission Types
 - **Read (R)** – Allows viewing the file
 - **Write (W)** – Allows modifying the file
 - **Execute (X)** – Allows running the file as a program
- Purpose of Permissions
 - Protect files from unauthorized access
 - Prevent accidental modification or deletion
- Example
 - Owner → Read & Write
 - Others → Read only

Common File Operations



Create

Creates a new file and allocates storage space

Open

Opens a file so a program can access it

Read

Retrieves data from the file

Write

Saves data into the file



Close

Closes the file after use

Delete

Removes the file and frees storage space

Rename

Changes the file name

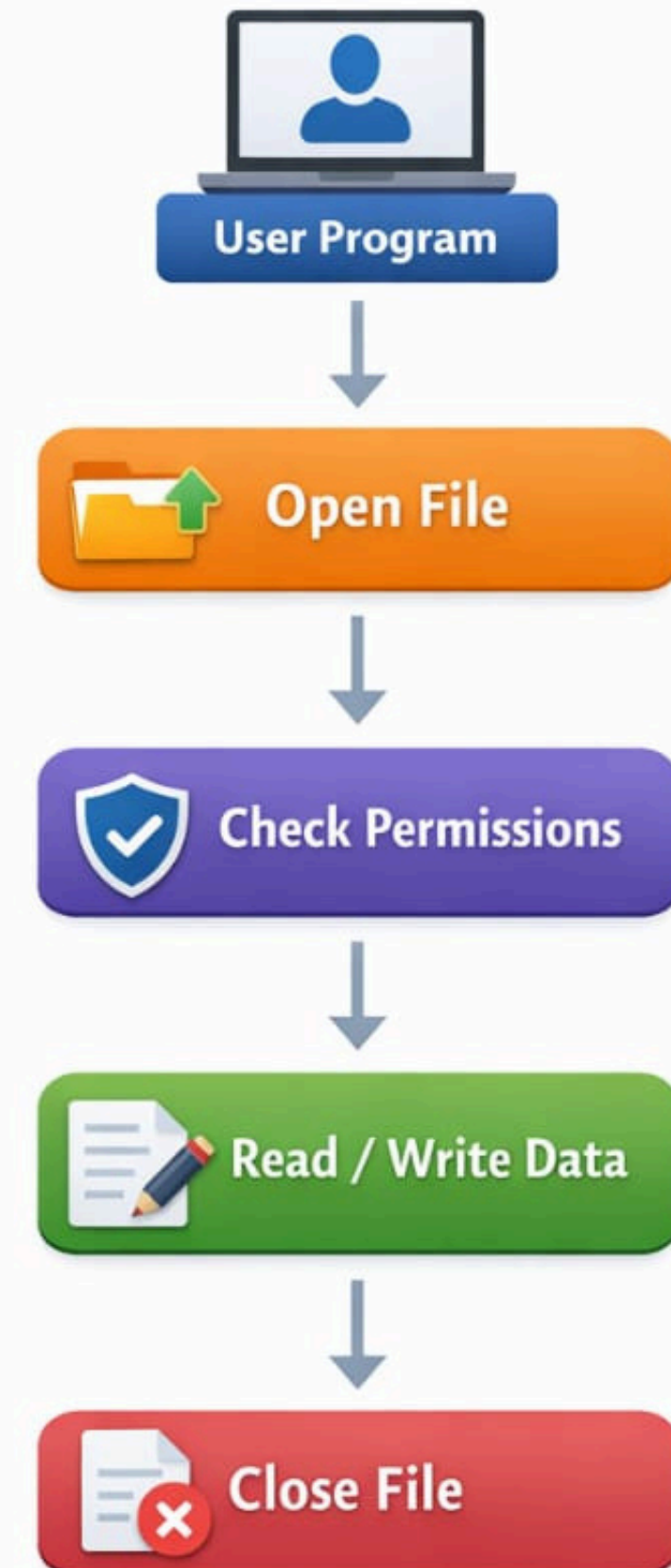
► File operations are actions performed by the Operating System on files.

► These operations are requested by programs through system calls.

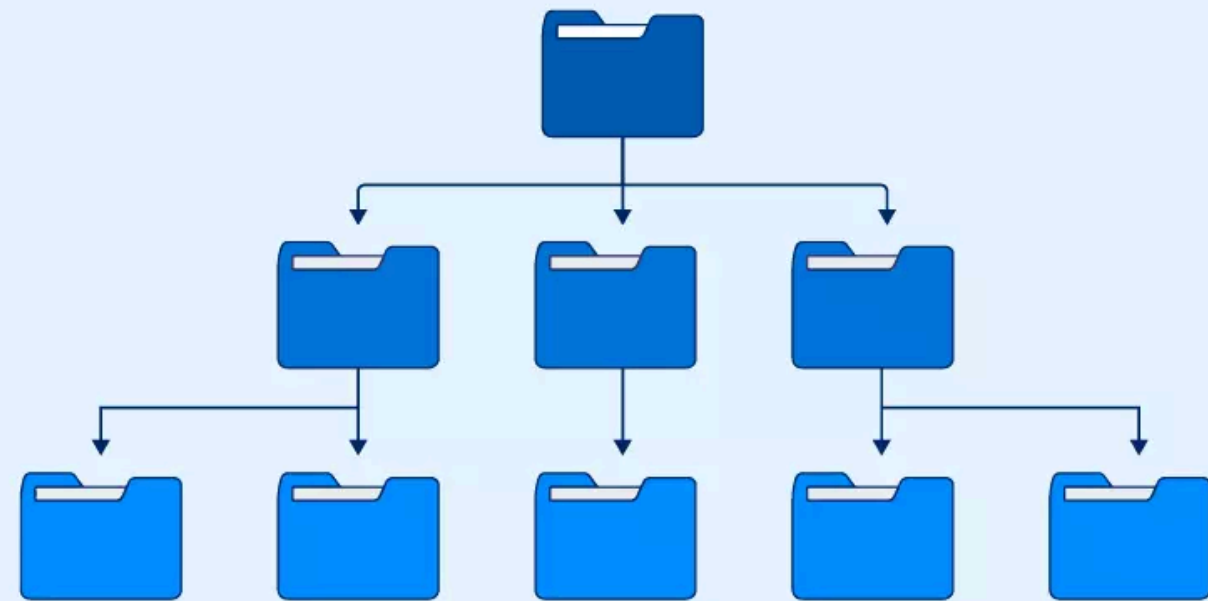
OS File Handling

- The Operating System uses structures such as:
 - File tables
 - Directory structures
 - Inodes (Unix/Linux systems)
- These store:
 - File metadata
 - File location on disk
- Programs use system calls to interact with the file system:
 - **open(), read(), write(), close(), delete()**

File Operation Flow



Directories



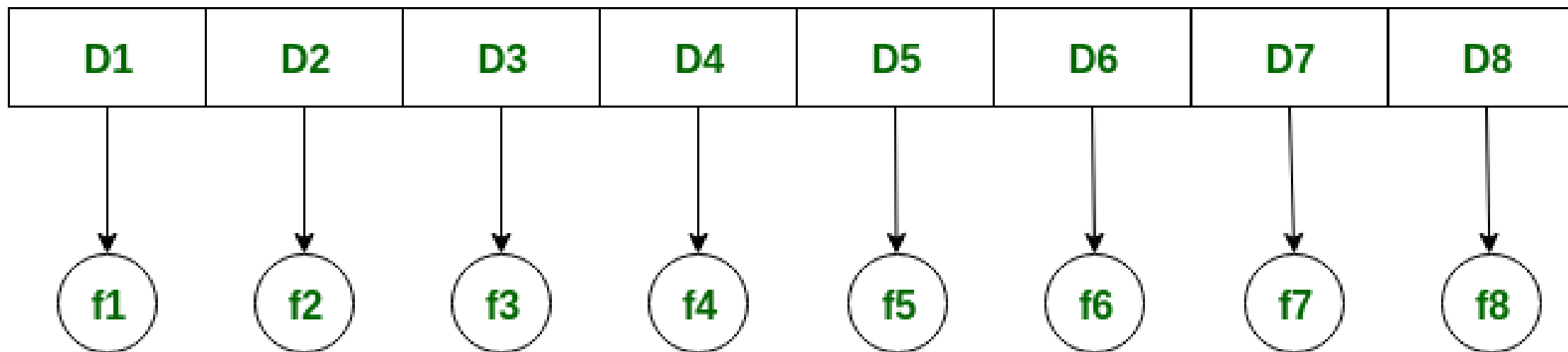
Directory Systems

- A **directory** is like a **folder** in which files are stored.
- It contains information about files, such as:
 - **File names**
 - **File locations** on disk
 - **File attributes**
(size, creation/modification dates, permissions)
- Directories can also contain **subdirectories**, forming a **hierarchical structure**.

Types of Directory Systems

1. Single-Level Directory

Directory



➡ All files are stored in one directory.

➡ Advantages:

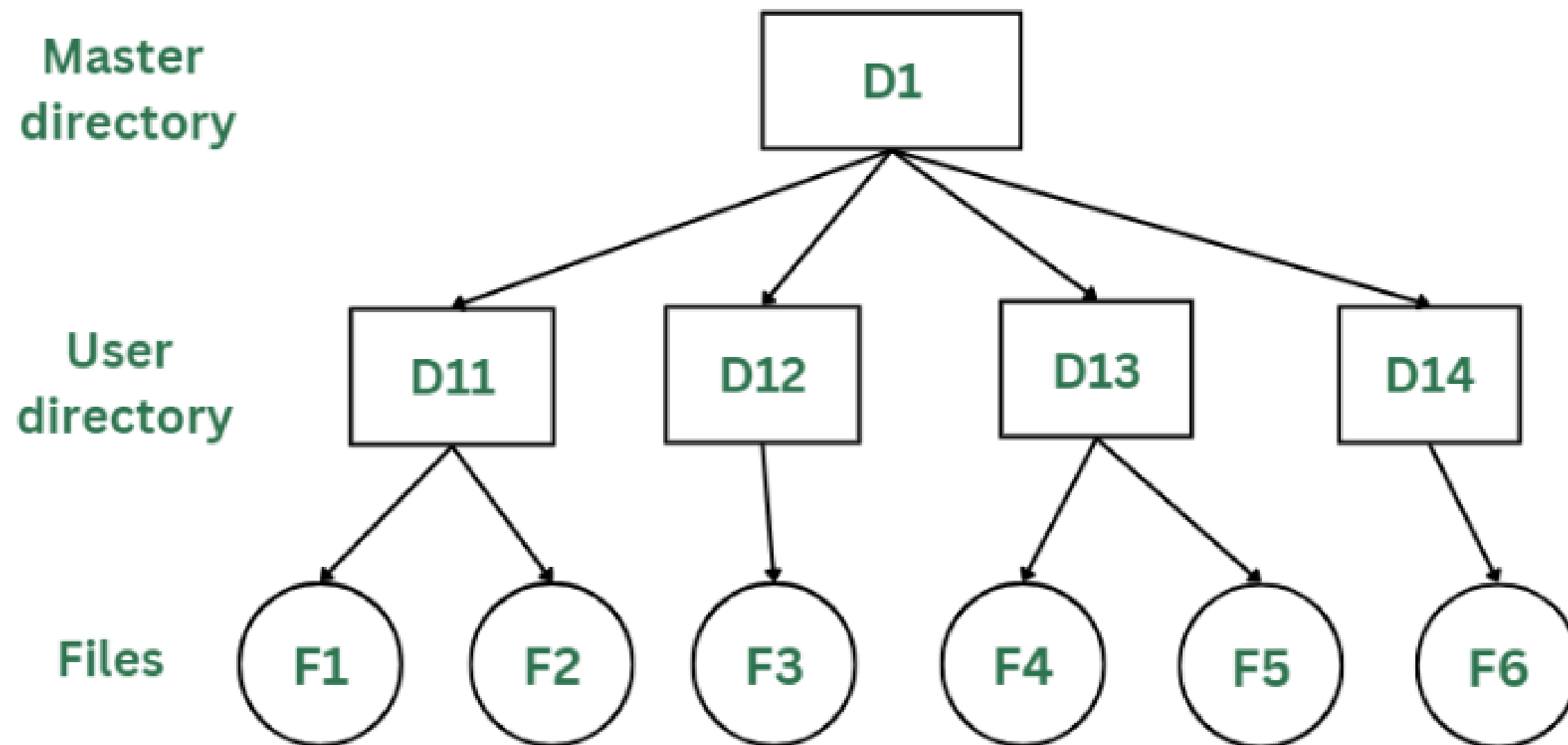
- Simple to implement.

➡ Disadvantages:

- File name conflicts are possible if two users want the same file name.
- Difficult to manage as the number of files increases.

Types of Directory Systems

2. Two-Level Directory



➡ Each user has a separate directory.

➡ Advantage:

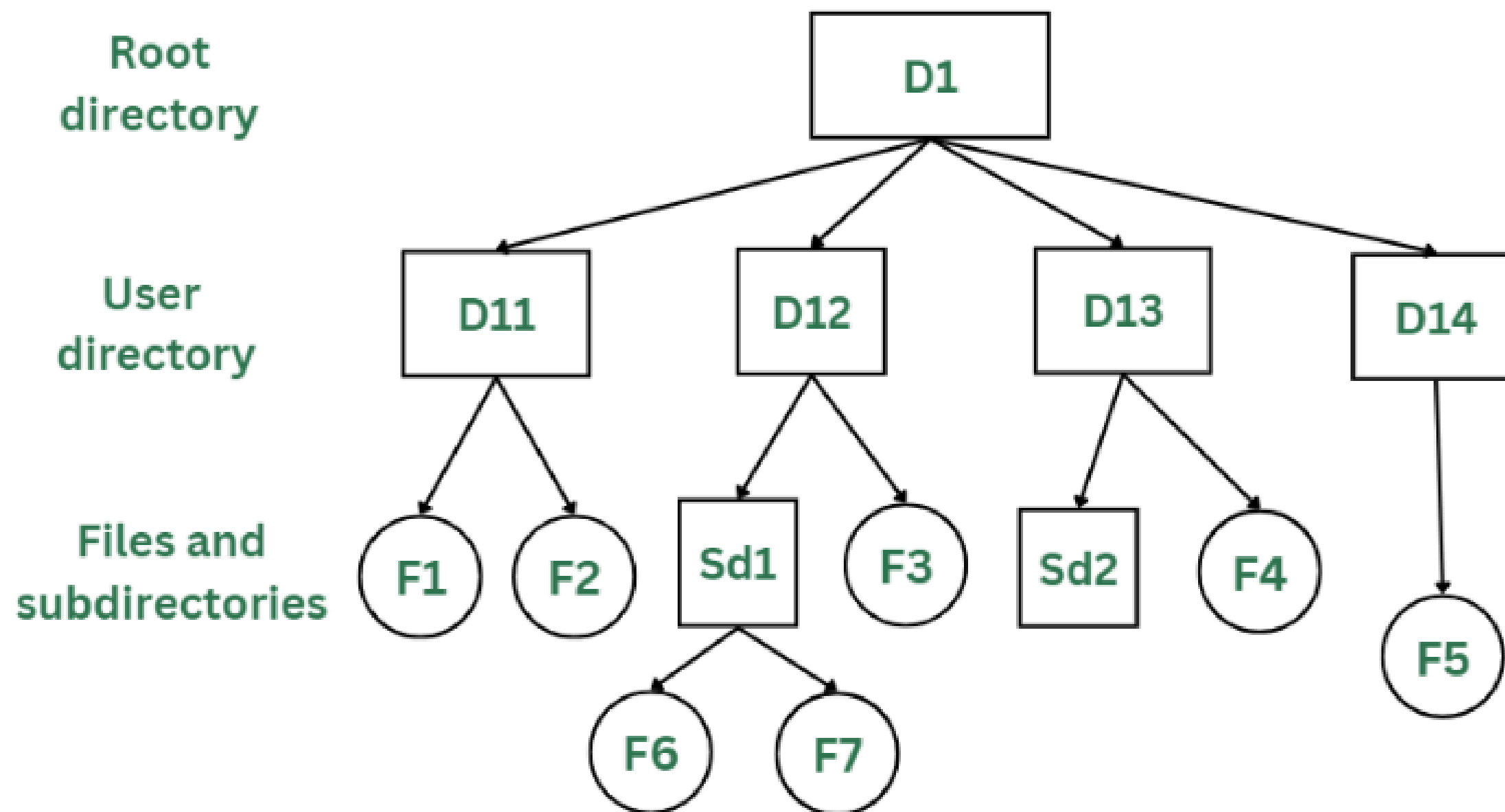
- Avoids file name conflicts between users.

➡ Disadvantage:

- Limited organization for complex file systems.

Types of Directory Systems

3. Tree-Structured Directory



► **Uses a hierarchical structure:**
directories contain subdirectories,
which contain files.

► **Advantages:**

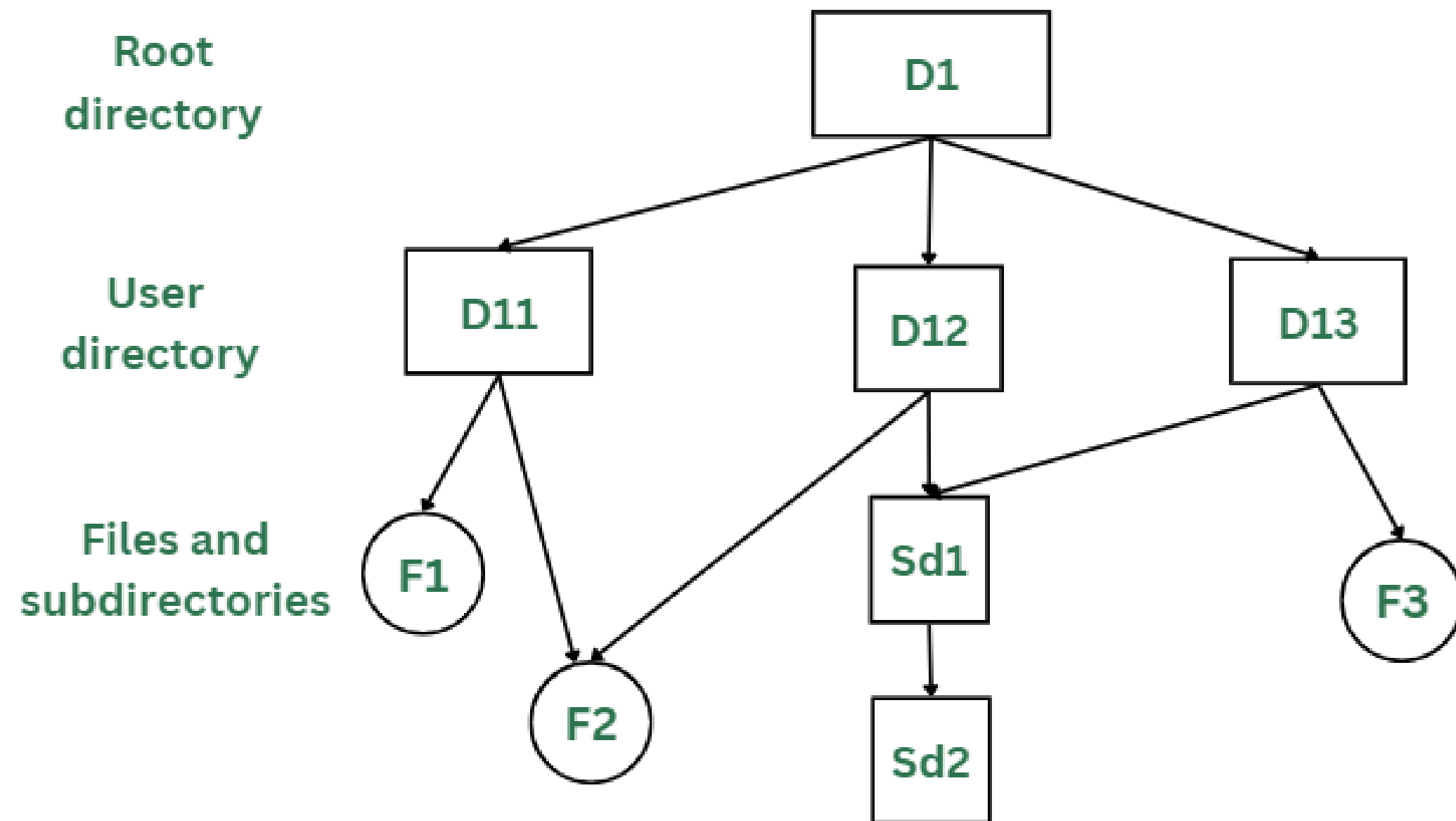
- Easy to manage and organize files.
- Commonly used in modern OS like Windows and Linux.

► **Disadvantages:**

- Slightly more complex to implement than single-level or two-level directories.

Types of Directory Systems

4. Acyclic Graph Directory



➡ Directories can share common subdirectories or files.

➡ Uses links to point to files in multiple locations.

➡ Advantages:

- Saves disk space by allowing shared files.

➡ Disadvantages:

- More complex to manage.

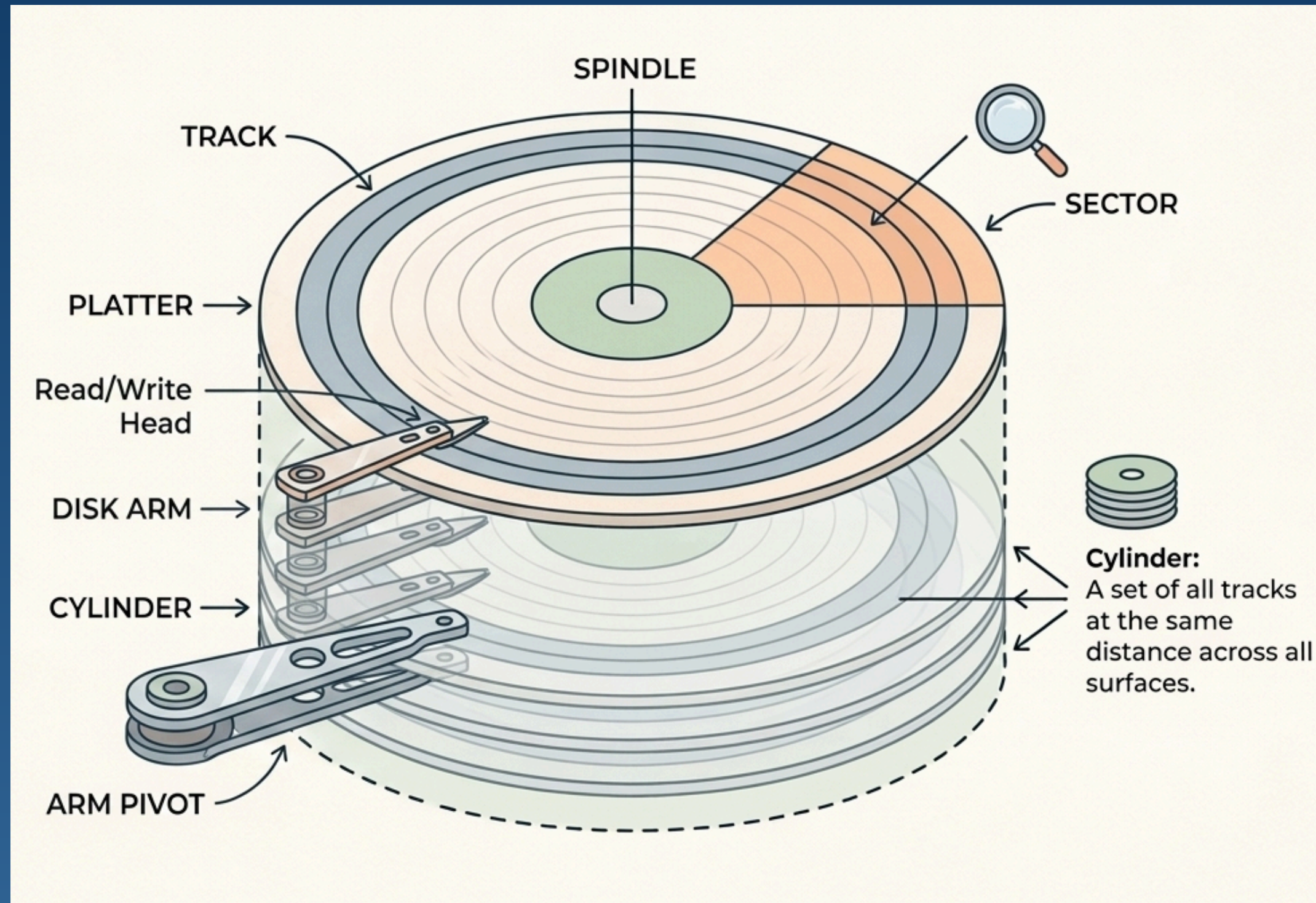
Disk Structure

- Disk storage is a **block device** that stores data in fixed-size blocks.
- Each block has a **unique address**, allowing independent reading and writing.
- Disks are organized into
 - **Tracks**
 - **Sectors**
 - **Cylinders.**



Disk Storage Management

Data Access Mechanism



- Data is accessed by moving the disk arm to the required track and waiting for the correct sector to rotate under the read/write head.
- This structure allows the operating system to store and retrieve files efficiently.

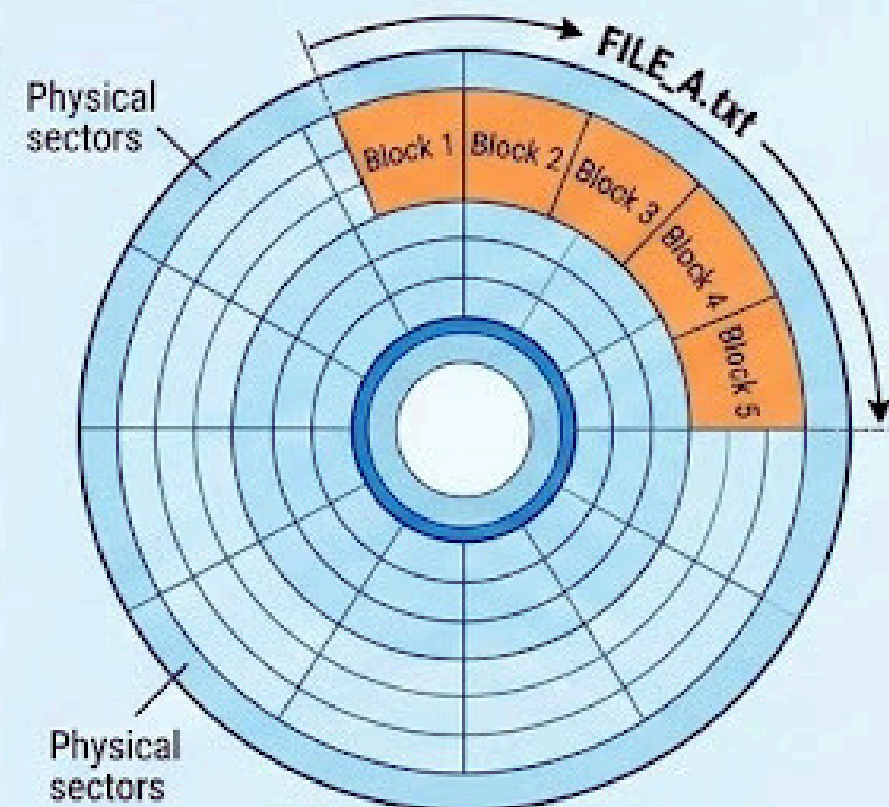
Logical vs Physical Storage

Feature	Logical Storage	Physical Storage
Perspective	How the OS and users view data.	How the hardware actually stores data.
Organization	Files, directories, and logical blocks.	Tracks, sectors, and cylinders.
Interface	Interacts with file names and paths.	Interacts with disk addresses and heads.
Management	Handled by the File System .	Handled by the Disk Controller .
Abstraction	Hides complex hardware details.	Deals with binary data and mechanical movement.

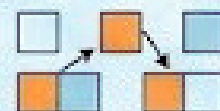
File Allocation Methods

FILE ALLOCATION METHODS IN OPERATING SYSTEMS

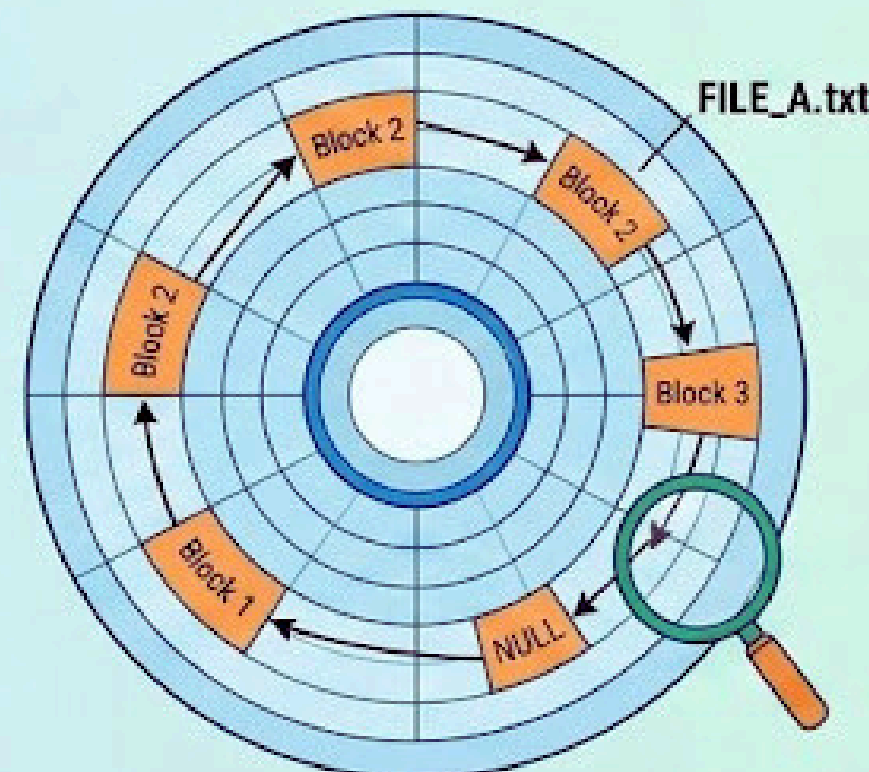
CONTIGUOUS ALLOCATION



- File is stored in continuous disk blocks.
- Provides fast access because blocks are next to each other.
- Can cause **external fragmentation** when free space becomes scattered.

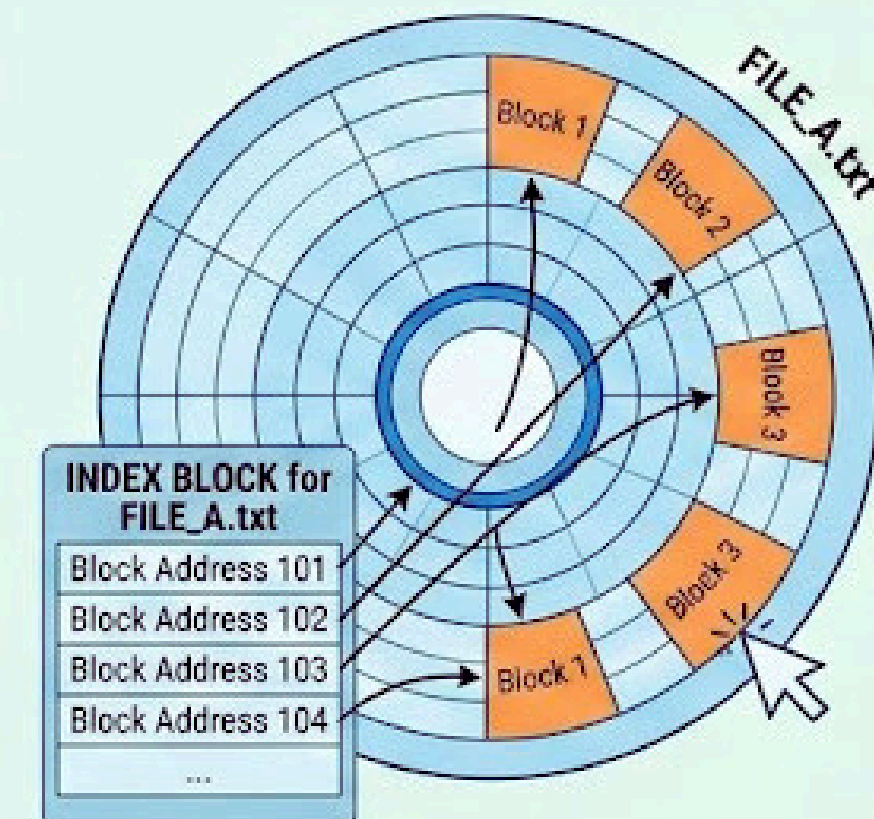


LINKED ALLOCATION



- File blocks are **linked together** using pointers.
- Blocks can be stored anywhere on the disk.
- **No external fragmentation**, but slower access since blocks must be followed sequentially.

INDEXED ALLOCATION



- Uses a special index block that stores the addresses of all file blocks.
- Allows direct access to any block of the file.
- Requires extra storage for the index block but improves access efficiency.

Free Space Management

- Free space management refers to how the **operating system keeps track of unused disk blocks.**
- The OS must know which blocks are **free and available for storing new files.**
- When a file is created, the OS **allocates free blocks** from this available space.
- When a file is deleted, its blocks are **returned to the free space list.**
- Efficient free space management helps reduce storage waste and improve disk performance.
- Common methods include **bit map (bit vector), linked list, grouping, and counting.**

File System Protection

- File system protection controls who can access, modify, or delete files.
- It prevents unauthorized access and data loss.
- Common protection methods:
 - **Access Control Lists (ACL)**
 - **File Permissions**
 - **Encryption**
 - **Authentication**
- Helps maintain data security, integrity, and privacy.



**Protection
&
Modern Files**

Modern File Systems

- Modern file systems are designed for **better performance, reliability, and security.**
- They support **large storage devices and large files.**
- Common examples:
 - **NTFS** – used in Windows, supports permissions and encryption.
 - **ext4** – widely used in Linux systems.
 - **APFS** – used in Apple devices.
- Features of modern file systems:
 - **Journaling** to prevent data corruption.
 - **Snapshot and backup support.**
 - **Better error recovery.**
 - **Efficient storage management.**

References

- **Andrew S. Tanenbaum. Modern Operating Systems (3rd ed.). Pearson Education.**
- **Silberschatz, A., Galvin, P. B., & Gagne, G. – Operating System Concepts.**
- **Stallings, W. – Operating Systems: Internals and Design Principles.**



Thank You

Group 11

